Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
«Вдосконалення структури коду графічного редактора об’єктів»

Виконала:

Студентка групи ІМ-55

Долгіх Дар’я Михайлівна
Номер у списку групи: 7
Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Лабораторна робота №4. Вдосконалення структури коду графічного редактора об’єктів

Мета: Мета роботи – отримати вміння та навички проектування класів, виконавши модернізацію коду графічного редактора в об’єктно-орієнтованому стилі для забезпечення зручного додавання нових типів об'єктів.

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab4.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Коди

Lab4.py

```
import os
import tkinter as tk
from shapes import PointShape, LineShape, RectShape, EllipseShape, LineOOShape,
CubeShape

class ToolTip:
    def __init__(self, widget, text):
        self.widget = widget
        self.text = text
        self.tip_window = None
        self.widget.bind("<Enter>", self.show_tip)
        self.widget.bind("<Leave>", self.hide_tip)

    def show_tip(self, event=None):
        if self.tip_window or not self.text:
            return
        x = self.widget.winfo_rootx() + 20
        y = self.widget.winfo_rooty() + 25
        self.tip_window = tw = tk.Toplevel(self.widget)
        tw.wm_overrideredirect(True)
        tw.wm_geometry(f"+{x}+{y}")
        label = tk.Label(tw, text=self.text, background="#ffffe0", relief=tk.SOLID,
borderwidth=1, font=("Tahoma", "8", "normal"))
```

```
        label.pack(ipadx=2, ipady=1)

    def hide_tip(self, event=None):
        tw = self.tip_window
        self.tip_window = None
        if tw:
            tw.destroy()

class MyEditor:
    def __init__(self, root):
        self.root = root
        self.current_tool = "Прямокутник"
        self.shapes = []
        self.max_shapes = 107
        self.current_shape = None

        self.tool_buttons = {}
        self.icons = {}

        self.shape_classes = {
            "Крапка": PointShape,
            "Лінія": LineShape,
            "Прямокутник": RectShape,
            "Еліпс": EllipseShape,
            "Лінія з кружечками": LineOOShape,
            "Куб": CubeShape
        }

        self.setup_menu()
        self.setup_toolbar()

        self.canvas = tk.Canvas(root, bg="white")
        self.canvas.pack(fill=tk.BOTH, expand=True)

        self.canvas.bind("<ButtonPress-1>", self.on_press)
        self.canvas.bind("<B1-Motion>", self.on_drag)
        self.canvas.bind("<ButtonRelease-1>", self.on_release)

        self.update_menu_and_title()

    def set_tool(self, tool_name):
        self.current_tool = tool_name
        self.update_menu_and_title()

    def update_menu_and_title(self):
        self.root.title("Лабораторна робота №4")
```

```
        tools = ["Крапка", "Лінія", "Прямокутник", "Еліпс", "Лінія з кружечками",
"Куб"]
        for idx, tool in enumerate(tools):
            prefix = "✓ " if tool == self.current_tool else "   "
            self.obj_menu.entryconfig(idx, label=f"{prefix}{tool}")

        for name, btn in self.tool_buttons.items():
            if name == self.current_tool:
                btn.config(relief=tk.SUNKEN, bg="#d0d0d0")
            else:
                btn.config(relief=tk.RAISED, bg="#f0f0f0")

    def setup_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)

        file_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Файл", menu=file_menu)

        self.obj_menu = tk.Menu(menubar, tearoff=0)
        tools = ["Крапка", "Лінія", "Прямокутник", "Еліпс", "Лінія з кружечками",
"Куб"]
        for tool in tools:
            self.obj_menu.add_command(label=tool, command=lambda t=tool:
self.set_tool(t))
        menubar.add_cascade(label="Об'єкти", menu=self.obj_menu)

        help_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Довідка", menu=help_menu)

    def setup_toolbar(self):
        toolbar = tk.Frame(self.root, bd=1, relief=tk.RAISED, bg="#f0f0f0")

        tools = [
            ("Крапка", "Малювання крапки", "point.png"),
            ("Лінія", "Малювання лінії", "line.png"),
            ("Прямокутник", "Малювання прямокутника", "rect.png"),
            ("Еліпс", "Малювання еліпса", "ellipse.png"),
            ("Лінія з кружечками", "Малювання лінії з кружечками", "line_oo.png"),
            ("Куб", "Малювання каркасу куба", "cube.png")
        ]

        current_dir = os.path.dirname(os.path.abspath(__file__))

        for name, hint, file_name in tools:
            img_path = os.path.join(current_dir, file_name)
```

```
            try:
                icon = tk.PhotoImage(file=img_path)
            except tk.TclError:
                icon = tk.PhotoImage(width=24, height=24)

            self.icons[name] = icon
            btn = tk.Button(toolbar, image=icon, command=lambda n=name:
self.set_tool(n), padx=4, pady=4, relief=tk.RAISED)
            btn.pack(side=tk.LEFT, padx=3, pady=2)
            ToolTip(btn, hint)
            self.tool_buttons[name] = btn

        toolbar.pack(side=tk.TOP, fill=tk.X)

    def redraw_all(self):
        self.canvas.delete("all")
        for shape in self.shapes:
            shape.draw(self.canvas)

    def add_shape(self, shape):
        if len(self.shapes) >= self.max_shapes:
            self.shapes.pop(0)
        self.shapes.append(shape)
        self.redraw_all()

    def on_press(self, event):
        shape_class = self.shape_classes.get(self.current_tool)
        if shape_class:
            self.current_shape = shape_class(event.x, event.y, event.x, event.y)

        if self.current_tool == "Крапка" and self.current_shape:
            self.add_shape(self.current_shape)
            self.current_shape = None

    def on_drag(self, event):
        if self.current_tool == "Крапка":
            self.add_shape(PointShape(event.x, event.y, event.x, event.y))
            return

        if self.current_shape:
            self.current_shape.x2 = event.x
            self.current_shape.y2 = event.y
            self.redraw_all()
            self.current_shape.draw_rubber(self.canvas)

    def on_release(self, event):
```

```
        if self.current_tool == "Крапка":
            return

        if self.current_shape:
            self.current_shape.x2 = event.x
            self.current_shape.y2 = event.y
            self.add_shape(self.current_shape)
        self.current_shape = None


main_window = tk.Tk()
main_window.geometry("800x600")
my_editor_global = MyEditor(main_window)

if __name__ == "__main__":
    main_window.mainloop()
```

shapes.py

```
class Shape:
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    def draw(self, canvas):
        pass

    def draw_rubber(self, canvas):
        pass


class PointShape(Shape):
    def draw(self, canvas):
        r = 4
        canvas.create_oval(
            self.x2 - r, self.y2 - r,
            self.x2 + r, self.y2 + r,
            fill="black", outline="black"
        )

    def draw_rubber(self, canvas):
        self.draw(canvas)
```

```
class LineShape(Shape):
    def draw(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2, fill="black")

    def draw_rubber(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2, dash=(4, 4),
fill="black")


class RectShape(Shape):
    def draw(self, canvas):
        canvas.create_rectangle(
            self.x1, self.y1, self.x2, self.y2,
            outline="black", fill=""
        )

    def draw_rubber(self, canvas):
        canvas.create_rectangle(self.x1, self.y1, self.x2, self.y2, dash=(4, 4),
outline="black")


class EllipseShape(Shape):
    def draw(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        canvas.create_oval(
            self.x1 - dx, self.y1 - dy,
            self.x1 + dx, self.y1 + dy,
            outline="black", fill="lightgreen"
        )

    def draw_rubber(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        canvas.create_oval(self.x1 - dx, self.y1 - dy, self.x1 + dx, self.y1 + dy,
dash=(4, 4), outline="black")


class LineOOShape(LineShape, EllipseShape):
    def draw(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        LineShape.draw(self, canvas)

        R = 3
```

```
        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + R, orig_y1 + R
        EllipseShape.draw(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + R, orig_y2 + R
        EllipseShape.draw(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2

    def draw_rubber(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        LineShape.draw_rubber(self, canvas)

        R = 3
        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + R, orig_y1 + R
        EllipseShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + R, orig_y2 + R
        EllipseShape.draw_rubber(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2


class CubeShape(RectShape, LineShape):
    def draw(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        shift_x = int((orig_x2 - orig_x1) * 0.3)
        shift_y = int((orig_y2 - orig_y1) * 0.3)

        self.x1, self.y1 = orig_x1 + shift_x, orig_y1 - shift_y
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        RectShape.draw(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x2, orig_y2
        RectShape.draw(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + shift_x, orig_y1 - shift_y
        LineShape.draw(self, canvas)
```

```
        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        LineShape.draw(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y1
        self.x2, self.y2 = orig_x2 + shift_x, orig_y1 - shift_y
        LineShape.draw(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y2
        self.x2, self.y2 = orig_x1 + shift_x, orig_y2 - shift_y
        LineShape.draw(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2

    def draw_rubber(self, canvas):
        orig_x1, orig_y1, orig_x2, orig_y2 = self.x1, self.y1, self.x2, self.y2

        shift_x = int((orig_x2 - orig_x1) * 0.3)
        shift_y = int((orig_y2 - orig_y1) * 0.3)

        self.x1, self.y1 = orig_x1 + shift_x, orig_y1 - shift_y
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        RectShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x2, orig_y2
        RectShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x1, orig_y1
        self.x2, self.y2 = orig_x1 + shift_x, orig_y1 - shift_y
        LineShape.draw_rubber(self, canvas)

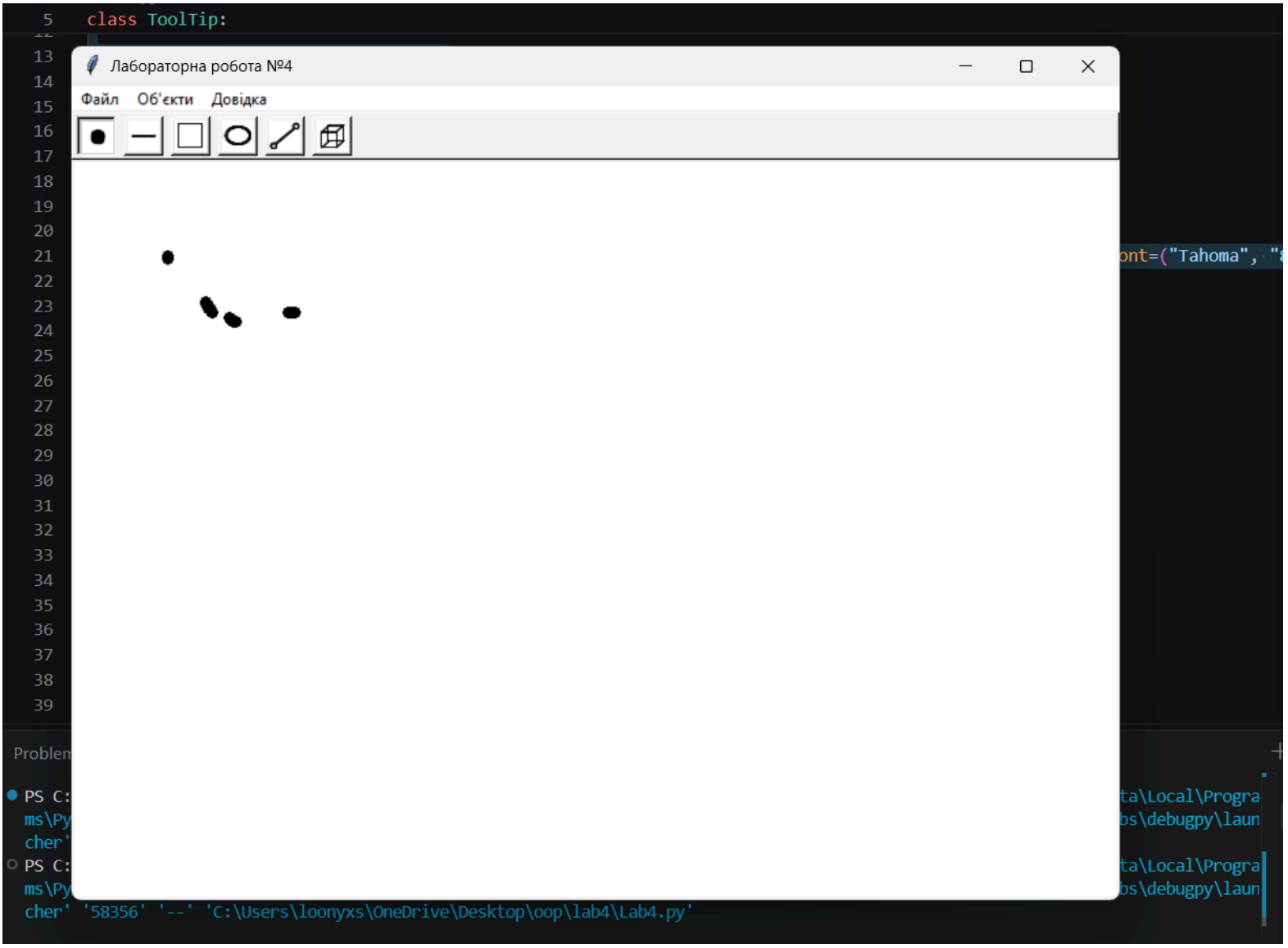
        self.x1, self.y1 = orig_x2, orig_y2
        self.x2, self.y2 = orig_x2 + shift_x, orig_y2 - shift_y
        LineShape.draw_rubber(self, canvas)

        self.x1, self.y1 = orig_x2, orig_y1
        self.x2, self.y2 = orig_x2 + shift_x, orig_y1 - shift_y
        LineShape.draw_rubber(self, canvas)

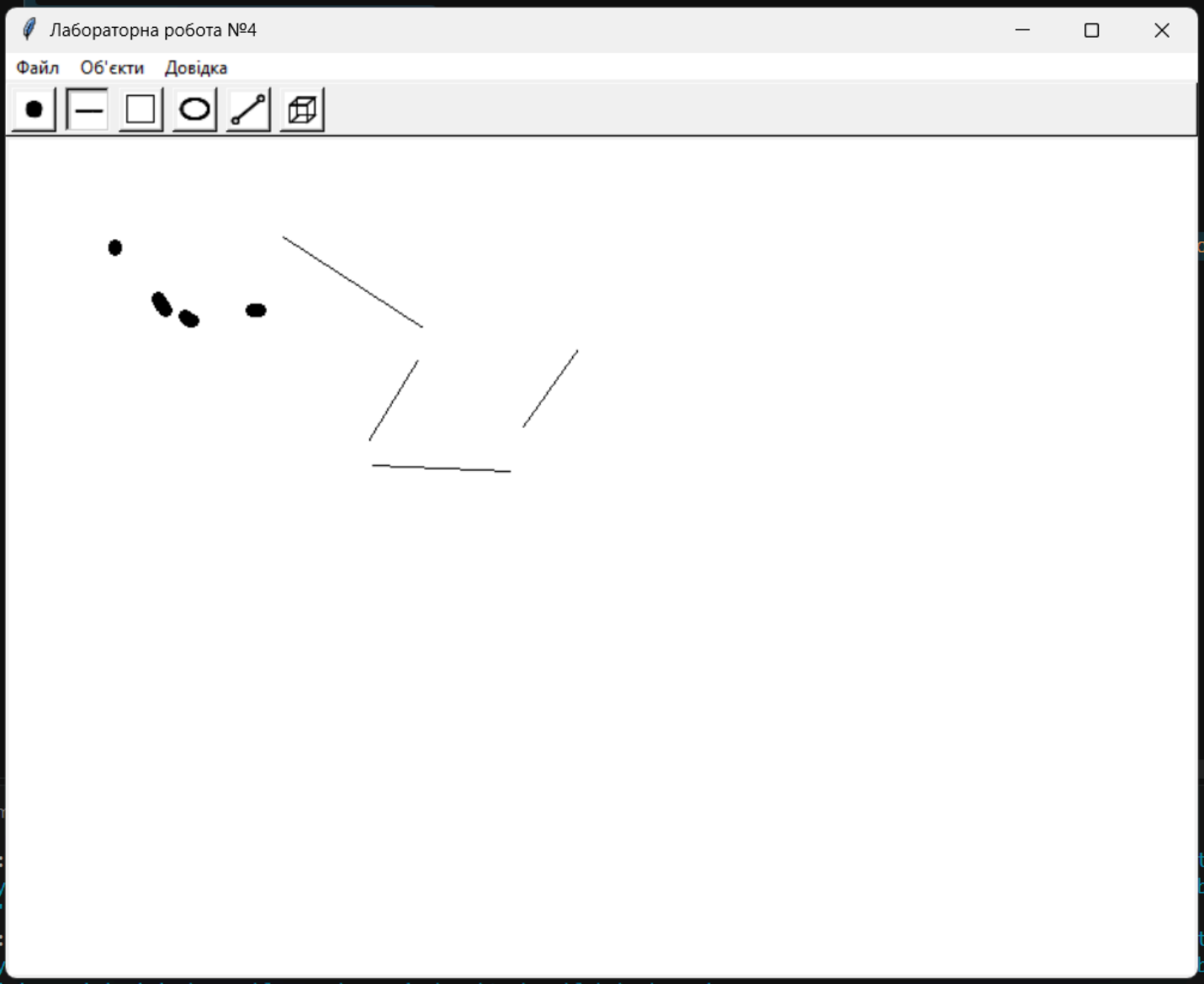
        self.x1, self.y1 = orig_x1, orig_y2
        self.x2, self.y2 = orig_x1 + shift_x, orig_y2 - shift_y
        LineShape.draw_rubber(self, canvas)

        self.x1, self.y1, self.x2, self.y2 = orig_x1, orig_y1, orig_x2, orig_y2
```

Виконання роботи



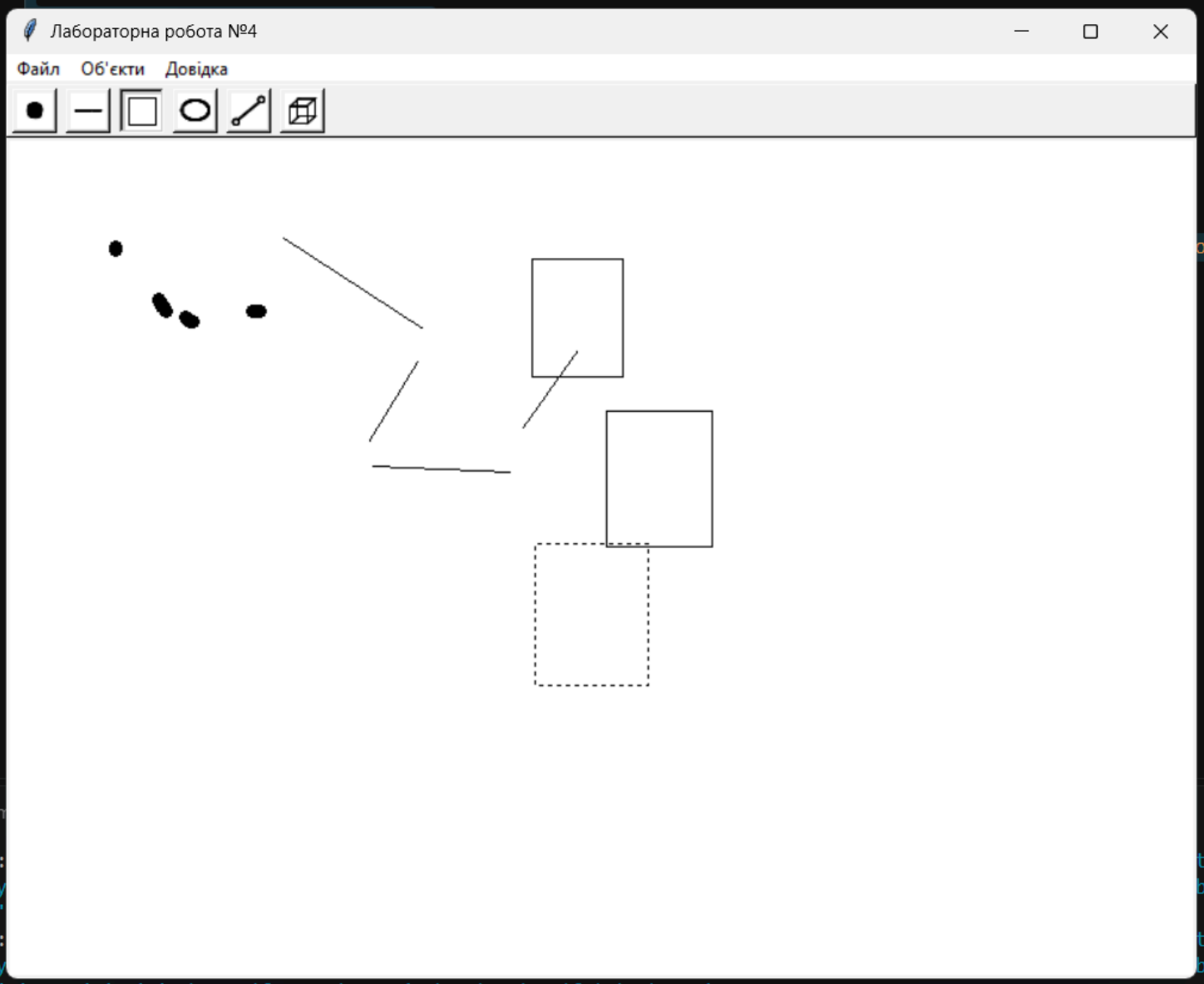
```
5   class ToolTip:
```



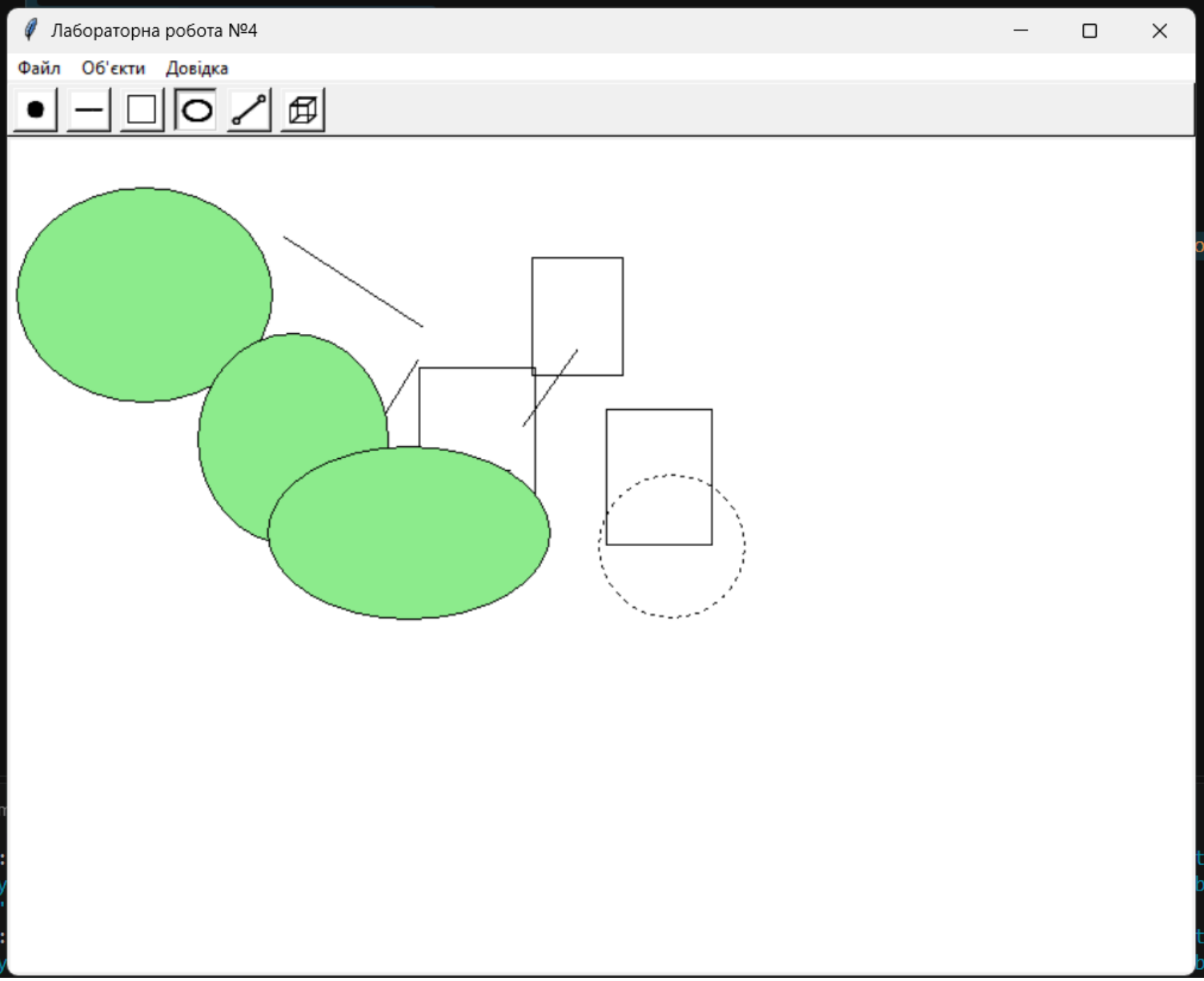

```
Problem

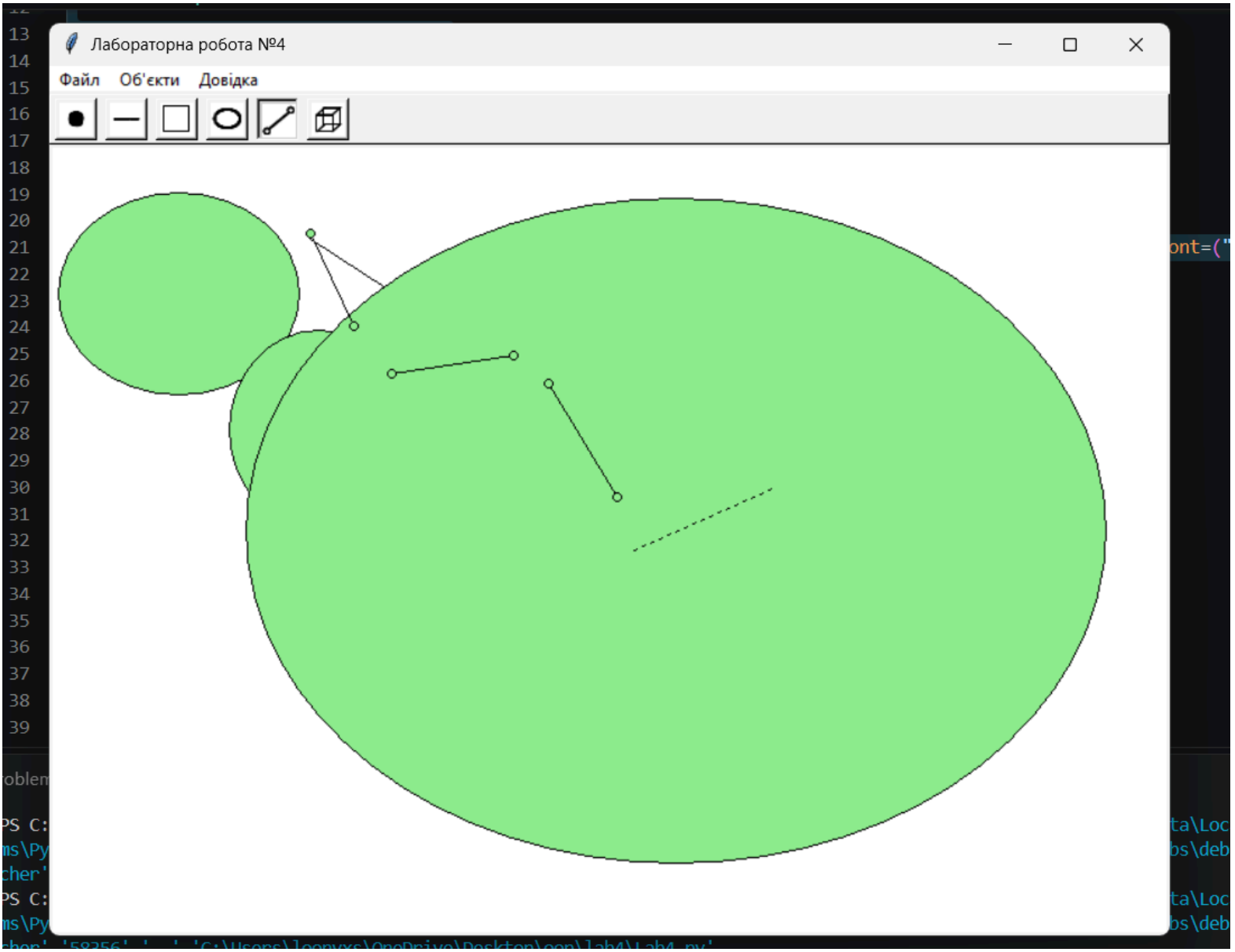
PS C:
ms\Py
cher'
PS C:
ms\Py
cher' '58356' '--' 'C:\Users\loonyxs\OneDrive\Desktop\oop\lab4\Lab4.py'
```

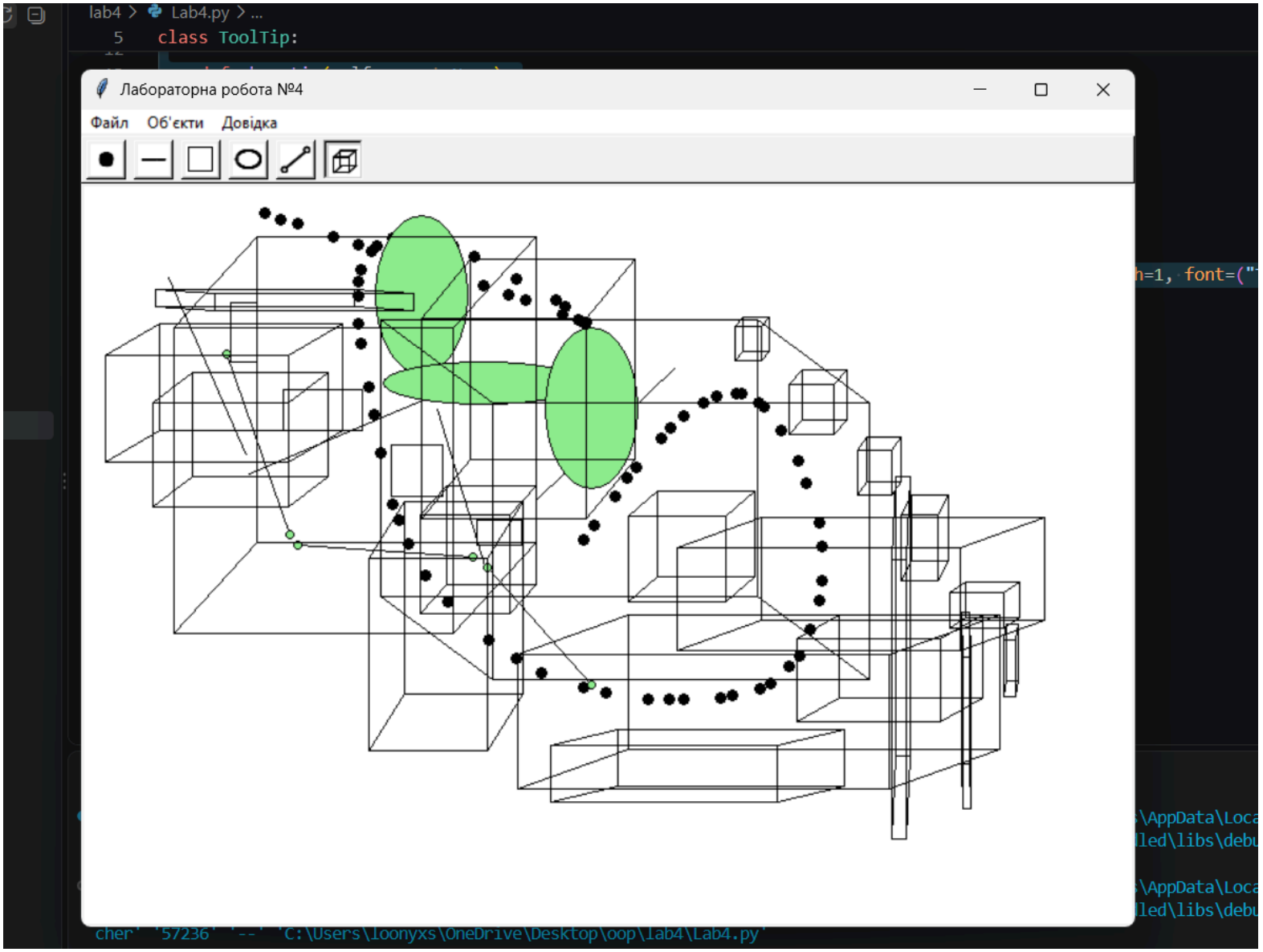
```
class ToolTip:
```



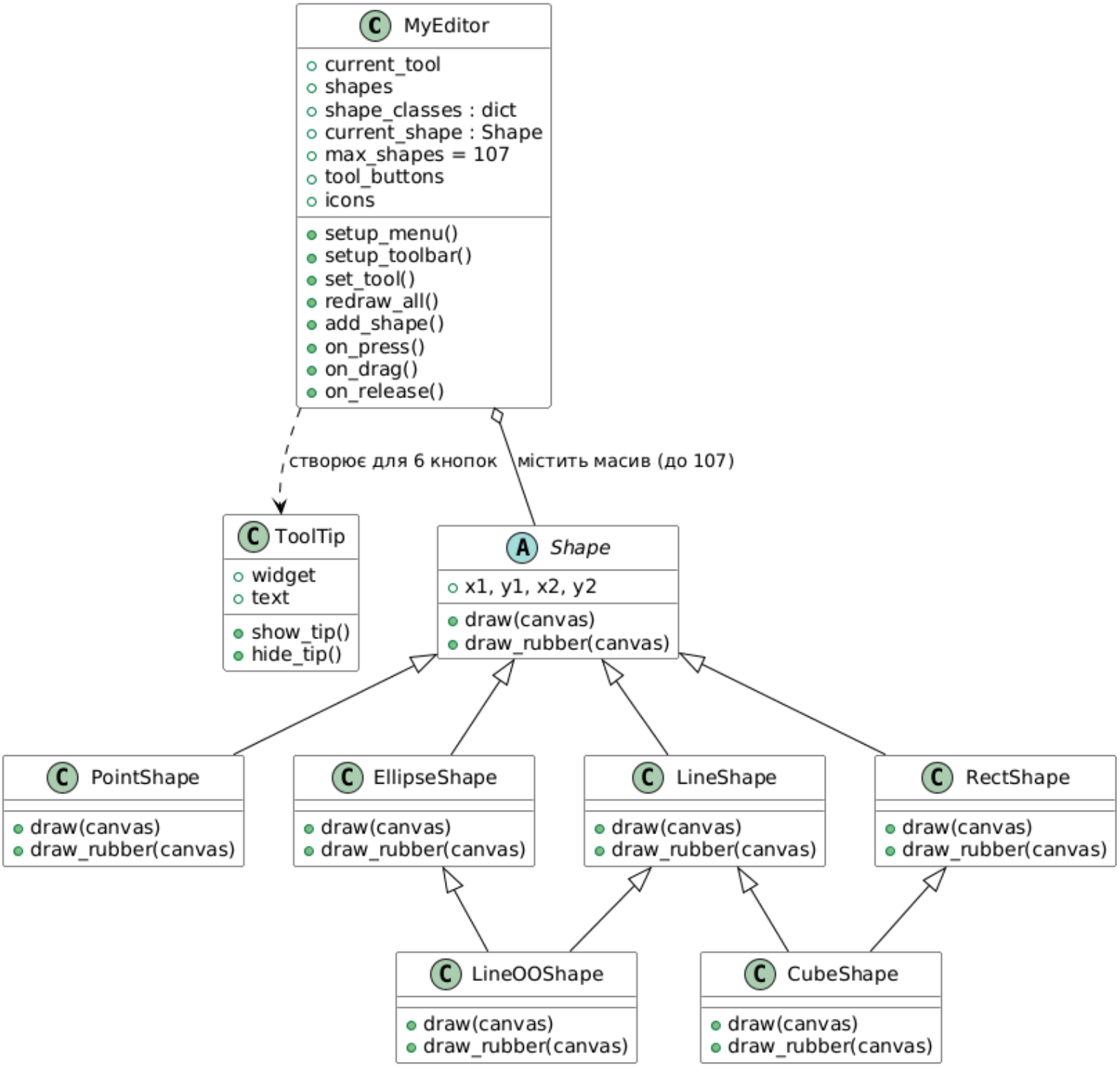

```
PS C:
ms\Py
cher'
PS C:
ms\Py
cher' '58356' '--' 'C:\Users\loonyxs\OneDrive\Desktop\oop\lab4\Lab4.py'
```


Лабораторна робота №4
Файл
Об'єкти
Довідка


Лабораторна робота №4
Файл
Об'єкти
Довідка


lab4 > Lab4.py > ...
5 class ToolTip:
Лабораторна робота №4
Файл
Об'єкти
Довідка
cher' '57236' '--' 'C:\Users\loonyxs\OneDrive\Desktop\oop\lab4\Lab4.py'

Схема



Висновок

Під час виконання лабораторної роботи №4 я закріпила навички об'єктно-орієнтованого проектування, виконавши модернізацію архітектури графічного редактора.

У ході роботи було застосовано поліморфізм: логіку малювання фігур та їхнього пунктирного "гумового" сліду перенесено всередину самих класів. За допомогою множинного успадкування реалізовано дві нові фігури («Лінія з кружечками» та

«Каркас куба»).Відповідно до непарного варіанту завдання, програму побудовано на базі глобального статичного об'єкта класу MyEditor. Також інтерфейс успішно доповнено панеллю Toolbar із графічними іконками та підказками (ToolTips). Мета роботи досягнута, програма відлагоджена та працює коректно.